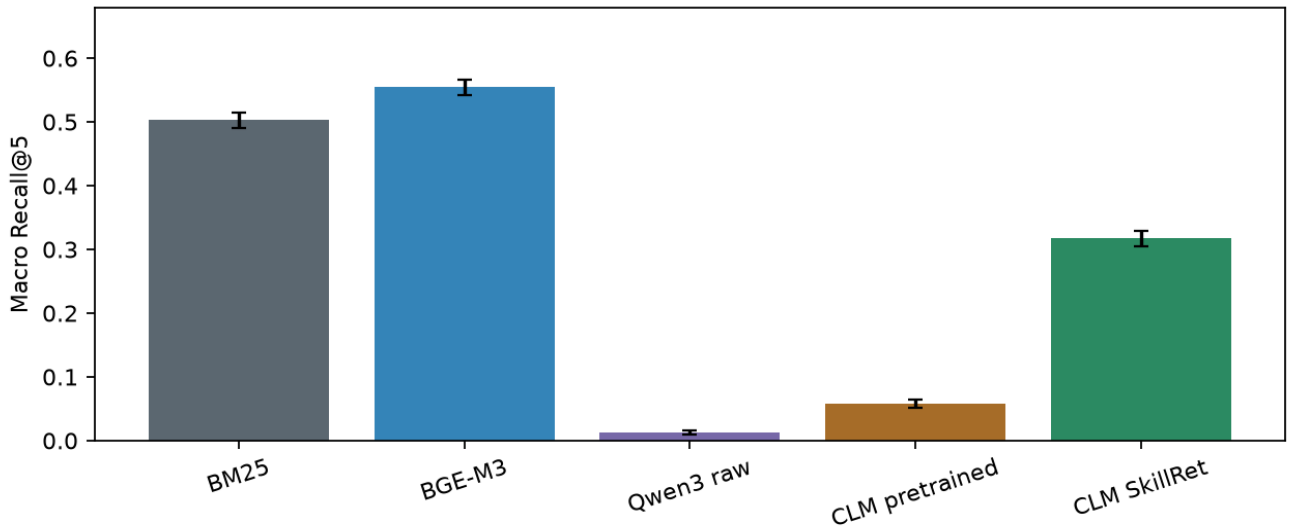
SkillRet full-catalog retrieval benchmark

Five fixed methods, description-only skill text, one official test catalog. Generated on RunPod 2026-09-28 11:44 UTC.

Method	Recall@5	95% group CI	MRR	Hit@1
BM25	0.5028	0.4896–0.5147	0.5701	0.4943
BGE-M3 dense	0.5545	0.5423–0.5660	0.6374	0.5651
Qwen3-8B pooled	0.0132	0.0101–0.0166	0.0146	0.0066
CLM pretrained	0.0582	0.0516–0.0648	0.0543	0.0253
CLM SkillRet adapted	0.3174	0.3045–0.3292	0.3151	0.2222

Observed adapted-minus-pretrained Recall@5: +0.2592. This is one training seed; the confidence interval resamples test query groups, not training randomness.



All five rows rank every test skill for every test query. No reranking shortlist or generative selector was used.

Protocol and data

Partition	Skills	Queries	Qrels
train	10123	63259	127190
test	6006	4392	7187

SkillRet dataset revision: 6583d7d2ed07644d0fb8938ed8178f3a7dc42a12. Query input is the canonical query string. Skill input is name, colon, space, description. The SkillRet query set is synthetic. Only train labels enter adaptation; test labels enter final evaluation after checkpoint selection.

BM25 uses rank-bm25 BM25Okapi with lowercased Unicode word tokens, k1=1.5 and b=0.75. BGE-M3 uses dense normalized embeddings. Qwen3-8B BF16 uses last-token pooling and L2 normalization before CLM heads. The pretrained and adapted CLM rows use the same frozen Qwen vectors.

Train and test skill IDs are disjoint at this dataset revision. ID disjointness does not establish semantic independence or completeness of relevance judgments. Duplicate texts retain distinct skill IDs.

Quality and paired comparisons

Method	Recall@1	Recall@10	NDCG@5	Complete@5
BM25	0.3405	0.5642	0.4741	0.3597
BGE-M3 dense	0.3994	0.6076	0.5310	0.4007
Qwen3-8B pooled	0.0052	0.0189	0.0097	0.0105
CLM pretrained	0.0216	0.0833	0.0422	0.0480
CLM SkillRet adapted	0.1782	0.3956	0.2669	0.2393

Pretrained CLM minus raw Qwen: +0.0451 Recall@5; paired 95% group interval [+0.0381, +0.0522].

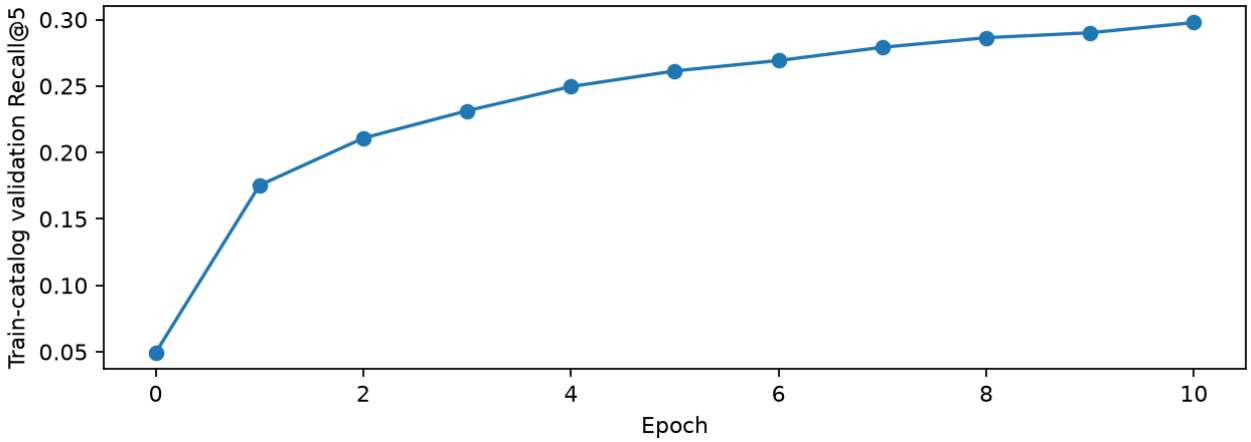
SkillRet CLM minus pretrained CLM: +0.2593 Recall@5; paired 95% group interval [+0.2472, +0.2712].

Hit@1 and Recall@1 differ when a query has multiple relevant skills. MRR uses the full ranking, not only top five. Exact saved score matrices and stable skill-ID tie breaking permit replay.

Head adaptation and validation

Training cohort: 49862 train query groups, with a separate 6309-query validation split. The cohort was selected from train groups before test scores were opened, using the measured Qwen pilot and budget guard.

Original epoch-0 validation Recall@5: 0.0493. Best actually trained epoch: 10, validation Recall@5 0.2978. Checkpoint choice used only train-catalog validation Recall@5, then MRR.



Only the released state/action projection heads and learned scale were updated. Each batch included all judged positives for its unique queries; the many-to-many loss also marked positives shared across queries. Unjudged pairs served as negatives, which is a limitation.

Timing and resource observations

Model	Fresh queries	p50 seconds	p95 seconds
bm25	20	0.144	0.220
bge_m3	20	0.148	0.193
qwen3	20	0.193	0.220

The latency sample includes fresh query encoding and full-catalog top-1 scoring with the model warm. It is a small sample. The BGE scoring path used CPU NumPy; encoder inference used the GPU. Qwen and both CLM rows share encoder work; the saved matrices are offline replay data and are not independent end-to-end latency measurements for the CLM rows.

BGE test skills: 6006 rows, 368749 tokenizer tokens, 21.4 seconds observed preprocessing.

BGE test queries: 4392 rows, 1275612 tokenizer tokens, 65.9 seconds observed preprocessing.

Full test scoring took 555.1s for BM25, 0.3s for BGE vectors and 0.5s for raw Qwen vectors. These matrix times exclude encoder loading and text encoding.

CLM projection of the full test queries and skills took 0.036s for pretrained heads and 0.050s for adapted heads.

Qwen process monitoring sampled 108 points: maximum total GPU memory 36,891 MiB; maximum attributed process RSS 2,212 MiB.

Interpretation, limits and reuse

This comparison supports statements about the pinned SkillRet synthetic queries, description-only skills, frozen encoders and one head-training seed. It does not establish universal retrieval quality or successful skill execution. BGE and BM25 differ from Qwen in architecture and pretraining, so only the Qwen → CLM → adapted CLM chain isolates head effects.

The archive includes every generated vector, ordered ID list, score matrix, checkpoint, split manifest, source pin, test evidence and scripts. The extracted best head requires the exact pinned Qwen3-8B encoder and input recipe to handle new text; the 8B backbone itself is intentionally not bundled.

Sources: ThakiCloud/SKILLRET dataset; BAAI/bge-m3; Qwen/Qwen3-8B; Contrastive-LM/CLM-v0.1-8B and pinned CLM source. Revisions and hashes are in the archive manifest.

RunPod billing is measured separately. A local cleanup receipt is appended to the final archive after the remote backup has been checked and the pod terminated.

RunPod credit at start: $7.5817; latest observed credit (2026-09-28T11:43:35.994034+00:00): $6.8744. These are billing observations before final pod cleanup, not an end-of-run invoice.

Observed retrieval misses

The examples below were chosen only after final evaluation. They are actual test queries and saved predictions; labels may omit semantically valid alternatives.

Shown examples are the first ASCII-renderable misses in the saved worst-case list.

Query q-02057: Can you go through this function and flag any spots where I've crammed too much logic into a single expression? I want a report that identifies each overly nested or chained inline expression, explains what it's doing, and suggests how to b

Gold: refactor-inline-complexity: Audit inline complexity and recommend variable extraction. Produces a re

Top predictions: commercial-lease-expert: Expert in commercial real estate lease agreements for industrial and office; legal-advisor: Draft privacy policies, terms of service, disclaimers, and legal

Query q-02754: Can you review this C# service class and refactor it? It's part of our "WarehouseOps" inventory management system and honestly it grew organically over the past year without much thought. I'm worried about SOLID violations, poor naming, and

Gold: clean-code-dotnet: Clean Code principles adapted for C#/.NET including naming, variables, functions,

Top predictions: blazemeter-troubleshooting: Comprehensive troubleshooting guide for BlazeMeter, covering API Monitor; Migration Planning: Systematic planning and execution of system migrations including database migrat

Reproduce the scores from the archived vectors with src/reproduce_from_vectors.py on a GPU. The released and adapted CLM heads are saved separately. The trained head can be loaded with src/load_clm_head.py using the pinned Qwen pre-head recipe in MODEL_CARD.md.